

Project Proposal Guide (Weeks 7–8)

Team Information

Project Title: RigMD: A Symptom-Guided Diagnostic and Decision Support System for Non-Technical Windows Desktop PC Users

Project Short Description (<20 words): A guided Windows desktop application that interprets user-reported PC performance symptoms against a system profile to deliver action-classified diagnostic advisory

Team Code: 2526-sem2-it332-11

Members:

1. Libato, Michael Grant
2. Macansantos, Axcelson
3. Ruperez, Raymart
4. Maestrado, Ralph Keane
5. Labaya, Godwin

PART 1: Introduction (Approx. 300 words)

Introduction

Desktop PC owners regularly encounter observable performance problems that they cannot independently interpret. A machine that fails to complete startup, freezes during normal use, produces repeated blue screen errors, generates driver conflict warnings, or runs noticeably slower than it previously did presents a problem with no immediately obvious resolution for users who lack a technical background. These are symptoms that manifest as Windows-observable behavior visible through system response patterns, Windows event logs, Device Manager status flags, S.M.A.R.T. storage health indicators, and driver conflict notifications, yet the ordinary desktop user has no unified tool to interpret what those signals

mean, what action to take next, or when the condition has exceeded the scope of what they can resolve themselves.

The challenge is not the absence of diagnostic tools. Brand-specific utilities such as MyASUS, HP Support Assistant, and Dell SupportAssist perform automated system health checks and driver advisory functions within their respective product ecosystems. Standalone tools such as HWiNFO64 and CrystalDiskInfo expose sensor readings and storage health data in considerable detail. Windows itself includes built-in troubleshooters for a range of common OS issues. Driver management applications such as DriverEasy provide advisory outputs for driver-related conflicts. The limitation shared by all of these is that they either operate within a single brand's ecosystem, address only one aspect of the diagnostic problem, or present their findings in technical language that requires expertise to act on. None of them functions as a decision-support tool — none classifies its findings into a defined action category, none measures whether the user left the interaction more confident in their next decision, and none tells the user clearly when a symptom has escalated to the point where professional inspection is the appropriate course of action. No existing consumer application integrates symptom-guided intake, OS-level reading, session history, action-classified advisory output, and plain-language decision support within a single brand-agnostic interface available to any Windows desktop user.

General-purpose AI tools can respond to symptom descriptions entered manually, but they maintain no persistent record of prior sessions, carry no awareness of the user's system profile or driver configuration, and produce no structured decision-support workflow connecting their responses to a defined action category or a measurable improvement in user decision confidence. Community forums provide a further alternative, but the guidance they offer is inconsistent, unverified, and rarely tailored to the exact system configuration of the person asking — and none of it measures whether the person asking was more confident in their decision after receiving it.

Research on AI-based decision support confirms that accuracy, transparency, structured output, and action classification are the primary determinants of whether non-expert users act on AI-generated guidance with confidence and make better decisions as a result (Ravi et al., 2025; Kostopoulos et al., 2024). Research on structured fault diagnosis further establishes that context-aware interpretation consistently outperforms unaided user judgment in technically demanding scenarios where system knowledge is required to act on findings (Zhang et al., 2019). These findings support the case for a diagnostic system designed explicitly as a decision-support tool — one that combines OS-level context, structured symptom intake, action-classified advisory output including a

professional inspection pathway, and AI-assisted plain-language explanation to produce guidance that measurably improves the individual desktop PC user's confidence in deciding what to do next.

Problem Statement

Desktop PC users who experience observable Windows-environment performance symptoms often lack a unified, brand-agnostic decision-support tool that connects their reported symptoms to their system profile, interprets the probable source of the issue, and classifies the result into clear next-step actions. Brand-specific diagnostic utilities are limited to their own product ecosystems. Standalone monitoring tools provide useful system readings but often require users to interpret technical data on their own. General-purpose AI tools can explain symptoms, but they do not maintain persistent system profile awareness, session history, recurring symptom tracking, or measurable evaluation of user decision confidence. Built-in Windows troubleshooters can address isolated issues, but they do not connect symptoms to a broader diagnostic pattern or provide a structured advisory record across sessions.

As a result, desktop PC owners, particularly students, gamers, beginner builders, budget users, and general home users, often rely on guesswork, scattered online advice, or raw technical readings they cannot confidently interpret. This may lead them to delay maintenance, apply incorrect troubleshooting steps, overlook recurring warning signs, or escalate a problem without fully understanding why. This study examines whether a structured, AI-assisted diagnostic decision-support application that combines system profile context, guided intake of Windows-observable symptoms, action-classified advisory output, session history, recurring symptom detection, and plain-language explanation can improve users' ability to understand the probable source of their PC problem and make a more confident next-action decision.

Research Gap

Prior research has addressed hardware failure prediction, symptom-based diagnosis, and AI-assisted decision support in clinical and enterprise contexts. However, limited work has examined how these concepts can be applied to a consumer-facing, brand-agnostic PC diagnostic decision-support system for individual Windows desktop users. Existing studies and tools often focus on technical diagnosis, system monitoring, or expert-level

troubleshooting, but they do not fully address how non-technical users can be guided toward a clearer and more confident next action.

Dahouk and Abu-Naser (2018) showed that rule-based troubleshooting systems can map reported PC symptoms to probable causes. However, their approach does not fully include persistent system profile context, session history, recurring symptom tracking, action-classified recommendations, or plain-language explanations designed for non-technical users. Existing consumer utilities such as MyASUS, HP Support Assistant, and Dell SupportAssist provide diagnostic and support features, but they are mainly limited to their own product ecosystems. Standalone tools such as HWiNFO64, CrystalDiskInfo, and Speccy can present useful system information, but they often require users to interpret raw technical readings on their own.

Research on explainable AI confirms that transparency, structured output, and action classification are essential to user trust and decision confidence in AI-generated guidance, but no existing consumer PC diagnostic tool has applied these decision-support design principles to Windows-observable, symptom-based internal diagnosis (Kostopoulos et al., 2024; Ravi et al., 2025). Context-based advisory systems have demonstrated that incorporating user profile data into interpretation logic substantially improves recommendation relevance, yet this principle remains unapplied in the consumer PC diagnostic domain for brand-agnostic, Windows-environment diagnostic guidance (Abu-Rasheed et al., 2023).

The gap this study addresses is the absence of a unified, consumer-accessible, brand-agnostic decision-support system that combines structured intake of user-reported, Windows-observable symptoms with persistent system profile context and session history, interpreted through an AI-assisted engine, to produce accurate internal diagnostic guidance, action-classified advisory outputs including a professional inspection category, and measurably improved user decision confidence — all within a single downloadable consumer application available to users regardless of their PC's manufacturer.

PART 2: Objectives

General Objective 1: Develop a Structured System Profile and OS Symptom Intake Module

To develop a structured system profile and Windows-observable symptom intake module that collects desktop PC system specifications and user-reported, Windows-observable performance symptoms through a guided diagnostic session, enabling the system to classify observable PC problems into defined symptom categories with at least 80% expert-validated intake relevance by the end of Semester 2, AY 2025–2026.

Specific Objectives:

- **1.1** To design a system profile input module that stores at least six key desktop PC specifications — CPU model, RAM capacity, storage type and capacity, operating system version, installed driver versions (GPU and chipset), and system age — to serve as the profile context for all subsequent diagnostic sessions.
- **1.2** To develop a structured diagnostic session form that collects at least eight symptom-related data points per session: symptom type, affected OS-level activity, symptom frequency, symptom severity, symptom duration, recent driver or software changes, system state at onset, and observable OS-level warning signs such as blue screen error codes, driver conflict notifications, Windows event log flags, repeated application crashes, or storage health alerts.
- **1.3** To implement a symptom classification mechanism that maps completed user responses to one of the following defined internal problem categories: OS performance issues, system stability issues, boot and startup issues, driver-related issues, storage OS-level issues, display and rendering issues, and no-power or unresponsive system triage.
- **1.4** To restrict the full diagnostic workflow to functional or partially functional desktop PCs exhibiting observable OS-level symptoms, while providing a safety-oriented boot-status triage checklist for fully non-functional units as the sole response for those cases, with a direct recommendation to seek professional inspection.
- **1.5** To enable users to submit accurate, structured symptom reports about their desktop PC without needing to know which technical details are diagnostically relevant, removing the barrier of technical knowledge from the first step of the diagnostic process and establishing the foundation for measurable improvement in user decision confidence from the earliest stage of the guided workflow.

General Objective 2: Develop a System-Aware Internal Diagnostic Interpretation Engine

To develop a system-aware internal diagnostic interpretation engine that analyzes the user's system profile, structured Windows-observable symptom responses, and prior session history to identify probable internal cause categories across OS performance, driver conflict, storage health behavior, and boot processes as primary diagnostic categories, achieving at least 80% expert-validated diagnostic output acceptance across a minimum of 20 controlled diagnostic scenarios by the end of Semester 2, AY 2025–2026.

Specific Objectives:

- **2.1** To implement a diagnostic mapping engine that connects Windows-observable symptom patterns to probable internal cause categories, designating OS performance degradation, driver conflicts, storage health behavior, and boot/startup failures as the four primary diagnostic targets across all supported symptom types.
- **2.2** To include supporting advisory categories — thermal condition as reported by OS sensors, display driver behavior, and system log event flags — for cases where symptom patterns extend beyond the four primary categories.
- **2.3** To use structured diagnostic rules combined with AI-assisted interpretation to generate plain-language diagnostic explanations comprehensible to non-technical desktop PC users, without overstating certainty or omitting the probabilistic basis of the output, so that users understand exactly why a particular finding and action category has been assigned to their case.
- **2.4** To prepare at least 20 controlled diagnostic test scenarios covering the following common desktop PC cases: slow boot, system freezing during multitasking, repeated blue screen errors, driver conflict behavior, storage OS-level warning indicators, application crashes following a system update, no-display behavior after a driver change, and basic no-power triage.
- **2.5** To ensure that users receive a clear, system-specific diagnostic result that identifies the probable source of their performance problem without requiring them to interpret raw technical readings themselves, making expert-level interpretation of Windows-observable symptoms accessible to non-technical desktop PC owners and directly improving their decision confidence compared to the pre-session baseline.

General Objective 3: Generate Advisory Recommendations, Track Session History, and Evaluate Decision Confidence

To generate system-aware advisory outputs that recommend practical next actions — including professional inspection when software-layer resolution is insufficient — implement a session-based history module that tracks recurring symptom patterns, and evaluate the system's usability and measurable impact on user decision confidence — achieving at least 80% expert-rated recommendation relevance across controlled diagnostic scenarios and a mean System Usability Scale score of at least 75 by the end of Semester 2, AY 2025–2026.

Specific Objectives:

- **3.1** To develop an advisory action module that classifies each recommendation into one of four defined action categories: Monitor, Maintain, Troubleshoot, or Escalate for Professional Inspection. The module will prioritize user-understandable OS-level guidance when appropriate and reserve escalation only for cases where the reported symptoms indicate that software-level troubleshooting may no longer be sufficient.
- **3.2** To ensure that every recommendation is accompanied by a plain-language explanation that explicitly connects the user's reported symptoms, system profile details, probable internal cause category, and the specific basis for the action category assigned — including a clear statement of why Escalate for Professional Inspection has been recommended when that category applies — together with a structured warning signs reference table specifying Windows-observable escalation thresholds and corresponding recommended actions.
- **3.3** To develop a session history module that saves each completed diagnostic session locally on the user's device — capturing the system profile snapshot, symptom inputs, diagnostic result, action category, confidence label, and timestamp — and flags recurring symptom types or internal cause categories when the same finding appears across two or more sessions associated with the same system profile, adjusting the advisory output and action category accordingly.
- **3.4** To conduct usability evaluation with at least 10 desktop PC users from the target population using the System Usability Scale instrument, and to administer a structured Likert-scale decision confidence survey before and after each participant's diagnostic session to measure change in self-reported confidence across four dimensions: understanding of the PC problem, identification of the likely internal cause, comprehension of the recommendation, and confidence in deciding on a next action — with a target of at least 25% improvement in composite decision confidence ratings from pre-session to post-session.

- **3.5** To enable users to receive a specific, actionable advisory output with a plain-language explanation and warning signs reference, review their diagnostic history to identify recurring problems, and interact with a system whose impact on decision confidence is measurable and empirically demonstrable — producing evidence that a structured, action-classified diagnostic decision-support workflow improves the quality of PC troubleshooting decisions among non-technical desktop users.

Research Questions:

1. To what extent does the structured Windows-observable symptom intake and system profile module collect relevant information needed to classify observable desktop PC problems into defined internal symptom categories?
2. How accurately does the system-aware internal diagnostic interpretation engine map user-reported, Windows-observable symptoms and system profile data to probable internal cause categories across OS performance, driver conflict, storage health behavior, and boot processes?
3. How relevant are the system-generated advisory recommendations and how usable is the diagnostic workflow, as measured by expert validation acceptance rates and the System Usability Scale, and to what extent does interacting with the system improve users' self-reported confidence in making PC troubleshooting decisions?

PART 3: Methods

Proposed Solution

The proposed system is a downloadable desktop application for Windows that functions as a decision-support tool, assisting individual desktop PC owners in identifying probable internal causes of Windows-observable performance symptoms and deciding on an appropriate, clearly classified next action. The system targets users who experience symptoms such as slow boot times, system freezing, repeated blue screen errors, driver conflict notifications, S.M.A.R.T. storage health alerts, application crashes, or display-related behavior reported through Windows — and who may not have sufficient technical

background to determine which internal component or system layer is responsible, or to decide whether the condition requires software-layer action or professional inspection. The user workflow follows four steps: the user creates a system profile, responds to structured symptom questions during a diagnostic session, receives an advisory output with an action-classified recommendation, plain-language explanation, and warning signs reference table on the result dashboard, and saves the session to their local history for future reference. The system is explicitly designed so that every completed session produces a measurable improvement in the user's decision confidence compared to their pre-session baseline.

The system is designed around a session-based diagnostic workflow rather than an open-ended conversational interface. This decision reflects two considerations. First, a structured workflow produces consistent, evaluable data across diagnostic sessions and enables reliable comparison with expert-validated test scenarios. Second, non-technical users who are unfamiliar with OS-level terminology are more likely to produce accurate and relevant symptom descriptions when guided through structured questions than when asked to compose free-text descriptions of problems they cannot fully characterize, and structured input produces output whose impact on decision confidence can be measured and validated.

The primary target users are students, gamers, beginner PC builders, budget desktop owners, content creators, and general home users — groups that commonly encounter Windows-observable performance symptoms but lack the technical background to interpret raw diagnostic readings and are therefore underserved by both brand-specific utilities and standalone monitoring tools. The system does not perform physical hardware evaluation or recommend component replacement or upgrades. All diagnostic output is bound to findings derived from user-reported, Windows-observable symptoms and system profile data. When symptoms indicate conditions that exceed software-layer resolution, the system produces a Escalate for Professional Inspection recommendation accompanied by a plain-language explanation of why software-layer action is insufficient — giving the user a justified, confident basis for that decision rather than an unexplained referral.

The system will be implemented as a Python desktop application using the CustomTkinter GUI framework for the Windows interface. Diagnostic rule logic will be implemented as a structured rule-based engine using Python conditional mappings between symptom inputs, system profile fields, and defined internal cause categories. Plain-language advisory explanations will be generated by calling the Claude API (claude-sonnet-4-6), with structured prompt templates that pass the classified symptom data and system profile context as inputs and constrain the output to a predefined advisory format. Session history

data will be stored locally on the user's device using SQLite via Python's built-in sqlite3 module, with no data transmitted externally beyond the API call for explanation generation. This technical approach is achievable within the semester timeline, does not require a server or network deployment, and produces a self-contained executable installable on any Windows desktop regardless of manufacturer.

The system's measurable outcome is the consolidation of eight diagnostic and advisory functions — each currently available only within separate brand-specific products or standalone tools — into a single brand-agnostic decision-support application accessible to any Windows desktop user, regardless of manufacturer, whose impact on user decision confidence is evaluated and reported as a primary outcome of the study.

Comparison of Similar Solutions

The following table presents a structured comparison between RigMD and eight existing products or tools that individually address one or more aspects of the desktop PC diagnostic problem. Each column represents a specific diagnostic or advisory function currently distributed across multiple separate tools. RigMD's position in the table represents the consolidation of all eight functions within a single brand-agnostic application, which constitutes the measurable outcome benefit to the user.

Comparison Feature / Benchmark	MyASUS	HP Support Assistant	Dell Support Assist	HWiNFO64	CrystalDisk Info	Windows Troubleshooter	DriveEasy	Specy	RigMD (Proposed System)
Brand-agnostic (works on any Windows desktop)	✗	✗	✗	✓	✓	✓	✓	✓	✓
Structured OS symptom intake with guided questions	✗	Limited	Limited	✗	✗	Limited	✗	✗	✓
Driver conflict	Limited	✓	✓	✗	✗	Limited	✓	✗	✓

detection and advisory									
Storage health (S.M.A.R.T.) OS-level interpretation	×	Limited	Limited	✓	✓	×	×	Limited	✓
Boot and startup issue detection	Limited	✓	✓	×	×	✓	×	×	✓
Session history and recurring pattern detection	×	×	×	×	×	×	×	×	✓
Plain-language advisory output with action classification	Limited	Limited	Limited	×	×	Limited	Limited	×	✓
Warning signs table presented with each advisory output	×	×	×	×	×	×	×	×	✓

Each of the eight functions listed in this table is drawn from an existing product or tool category. MyASUS contributes the model of automated system health advisory; HP Support Assistant and Dell SupportAssist contribute driver and boot diagnostic flows; HWiNFO64 and CrystalDiskInfo contribute hardware sensor and storage health reading; Windows Troubleshooter contributes OS-level automated issue detection; DriverEasy contributes driver-focused advisory; and Speccy contributes system profile reading for context. RigMD's benchmark is defined by this comparison: the system's success is evaluated against its ability to deliver all eight functions within a single application at or above the performance thresholds specified in each objective.

Warning Signs Reference Table (Post-Recommendation)

The following table is presented to the user on the result dashboard following each advisory recommendation. It specifies the observable OS-level warning signs associated with the diagnosed category, the threshold at which each sign warrants escalation, and the

corresponding recommended action. This table is generated as part of the advisory output, is tailored to the symptom category identified during the diagnostic session, and is designed to give the user a clear, concrete basis for monitoring their own system and making a confident decision about when to escalate.

Warning Sign	Observed Threshold	Recommended Action
Repeated blue screen error codes (same code)	2 or more occurrences within 7 days	Troubleshoot: identify and roll back the associated driver or recent system update
Windows Event Log critical errors (System log)	3 or more critical events logged within 48 hours	Troubleshoot: review event source and initiate system file check
Storage health status downgraded (S.M.A.R.T. OS alert)	Any status change from Good to Caution or Bad	Maintain: back up user data immediately; monitor for further degradation
Driver conflict notification (Device Manager yellow flag)	Any active conflict flag on primary system devices	Troubleshoot: update, roll back, or reinstall affected driver
Startup repair prompt at boot	Any occurrence	Troubleshoot: run Startup Repair; escalate to professional help if unresolved
System freeze recurring during the same activity	2 or more occurrences within the same session type	Monitor: log the activity and escalate to Troubleshoot if frequency increases
No display output after OS login	Persistent across two or more reboots	Escalate for Professional Inspection: condition exceeds OS-level resolution
Windows Update failure repeated	3 or more failed update attempts	Troubleshoot: run Windows Update troubleshooter; check disk space and connectivity

Development Methodology

The system will be developed using the Agile methodology, organized into focused, time-boxed development sprints. Agile is appropriate for this project because the application

consists of discrete and independently testable modules: the system profile input module, the structured OS symptom intake module, the internal diagnostic interpretation engine, the advisory action module, the transparent result dashboard with warning signs table, and the session history and recurring pattern detection module. Each module can be built, internally tested, and reviewed against its corresponding objective before integration with the rest of the system.

At the conclusion of each sprint, the completed module or feature increment will be reviewed against the specific objectives it is intended to fulfill. Feedback from this review will inform the scope of subsequent sprints. This iterative approach accommodates refinements identified during development, ensures each module functions correctly in isolation before integration testing begins, and maintains alignment between the evolving prototype and the proposal's documented objectives throughout the development period.

Validation Approach

System validation will proceed through three complementary methods: expert validation of diagnostic and recommendation outputs, usability testing with target users, and qualitative feedback collection.

For expert validation, a minimum of 20 controlled diagnostic scenarios will be prepared before evaluation. Each scenario will specify a sample system profile, a description of user-reported, Windows-observable symptoms, an expected probable internal cause category, and an expected advisory action. These scenarios will cover the full range of supported symptom types, including slow boot, system freezing, blue screen errors, driver conflict behavior, Windows-reported storage health warnings, repeated application crashes, no-display behavior after a driver change, and cases expected to route to the Escalate for Professional Inspection category. At least two technically knowledgeable evaluators will independently assess each system-generated output using a structured rubric measuring diagnostic relevance, recommendation suitability, explanation clarity, system profile compatibility, and safety appropriateness. The minimum threshold for overall expert acceptance is 80% across all evaluated scenarios.

For usability evaluation, at least 10 desktop PC users representing the target population will interact with the installed application by creating a system profile, completing at least one full diagnostic session, reviewing the advisory output and warning signs table on the result dashboard, and completing the evaluation instruments. The System Usability

Scale will be administered following the session. A structured Likert-scale survey measuring self-reported decision confidence across four dimensions — understanding of the PC problem, identification of the likely internal cause, comprehension of the recommendation, and confidence in deciding on a next action — will be administered both before and after the session. The target is a mean SUS score of at least 75, exceeding the established SUS benchmark of 68 for acceptable usability (Brooke, 1996), and a minimum 25% improvement in composite decision confidence ratings from pre-session to post-session. Decision confidence improvement is treated as a primary outcome of the study, not a secondary metric.

Qualitative data will be gathered through brief semi-structured interviews with usability participants following the evaluation survey. Interview responses will be analyzed thematically around five areas: perceived clarity of the diagnostic explanation, trust in the advisory recommendation, ease of navigating the guided workflow, usefulness of the warning signs table, and perceived value of the session history feature.

Quantitative data will be summarized using descriptive statistics. Expert validation acceptance rates will be reported as percentages by scenario and overall. System Usability Scale scores will be averaged across participants. Pre-session and post-session confidence ratings will be compared and the percentage change calculated and reported as the study's primary human impact finding.

PART 4: Expected System

The proposed system will be a downloadable desktop diagnostic decision-support application for Windows. Its purpose is to help individual desktop PC owners understand the probable internal causes of Windows-observable performance symptoms, identify an appropriate and clearly classified next action through a guided session-based workflow, and leave each session measurably more confident in their troubleshooting decision than they were before it began. After installing the application, the user creates or confirms a system profile, responds to structured symptom questions during a diagnostic session, reviews a structured advisory output on the result dashboard, consults the warning signs reference table, and saves the session for future reference. AI assistance is used within this workflow to generate plain-language explanations of the diagnostic result, but the overall process is

governed by structured input fields, predefined symptom categories, expert-validated diagnostic logic, and four defined action categories rather than open-ended generation.

The minimum viable product consists of six integrated features.

Feature 1: System Profile Module

The system profile module stores the user's desktop PC specifications — CPU model, RAM capacity, storage type and capacity, operating system version, installed driver versions for GPU and chipset, and system age — as the profile context for all associated diagnostic sessions. Depending on what is technically feasible within the prototype, the system profile may be entered manually through guided input forms, partially assisted through auto-detection of local system information using Python's platform and wmi libraries, or a combination of both.

The system profile is foundational to the system's value as a decision-support tool. Without it, any advisory output the system generates is necessarily generic and cannot produce a decision the user can act on with confidence. With it, the system can produce recommendations that are relevant to the user's actual configuration — recognizing, for example, that a driver conflict advisory appropriate for a system running an outdated GPU driver may be entirely inappropriate for a system whose driver is current and whose Windows-observable symptoms point instead toward a storage health issue.

Feature 2: Structured OS Symptom Intake Module

The structured symptom intake module guides the user through a diagnostic session using controlled questions rather than free-text input. The module collects at least eight data points per session: symptom type, affected Windows-environment activity, frequency, severity, duration, recent driver or software changes, system state at the time the symptom occurs, and Windows-reported warning signs. Affected activities may include startup, application use, gaming, multitasking, file access, video editing, heavy computation, or idle use. Warning signs may include blue screen error codes, driver conflict notifications, Windows event log alerts, S.M.A.R.T. storage health status changes, repeated application crashes, or no-display behavior.

Structured symptom intake serves two purposes. For the user, it removes the burden of knowing which details are diagnostically relevant and begins the process of building toward a confident decision from the first question. For the system, it ensures that every

diagnostic session produces a consistent, evaluable data set that can be reliably classified and compared against expert-validated test scenarios.

Feature 3: Internal Diagnostic Interpretation Engine

The internal diagnostic interpretation engine analyzes the system profile, user-reported Windows-observable symptom responses, and prior session history to identify a probable internal cause category. The four primary diagnostic categories are OS performance degradation, driver conflict, storage health behavior, and boot and startup failure. Supporting categories — thermal condition as reported by Windows sensors, display driver behavior, and system log event flags — are included for cases where the symptom pattern extends beyond the primary categories.

Outputs are framed as probable advisory findings rather than confirmed diagnoses. For example: "The reported slow boot times, repeated startup repair prompts, and observed storage health warning indicator suggest a probable storage OS-level issue. Backing up important files is the recommended first action before any further troubleshooting is initiated." This framing accurately reflects the probabilistic nature of symptom-based internal diagnosis, manages user expectations regarding the certainty of the result, and equips the user with the reasoning they need to act on the finding with confidence.

Feature 4: Advisory Action Module

The advisory action module converts the diagnostic interpretation into a practical recommended next step, classified into one of four defined action categories: Monitor, Maintain, Troubleshoot, or Escalate for Professional Inspection. This four-category classification structure is the core of the system's function as a decision-support tool — it ensures that every session ends with a concrete, bounded decision the user can act on with confidence, not an ambiguous status reading they must interpret themselves.

If symptoms are mild and inconsistent, the system recommends monitoring. If symptoms suggest dust-related thermal behavior or software bloat affecting performance, the system recommends maintenance. If symptoms followed a recent driver update, software change, or system update, the system recommends troubleshooting before any escalation. If symptom patterns involve conditions that exceed software-layer resolution — such as no-power behavior, persistent no-display output across multiple reboots, or repeated startup failures unresolved by available Windows tools — the system recommends seeking professional help, accompanied by a plain-language explanation of why software-layer action is no longer the appropriate path. Every recommendation across all four categories is

accompanied by a plain-language explanation generated via the Claude API and a warning signs reference table specifying Windows-observable escalation thresholds and corresponding actions.

Feature 5: Transparent Result Dashboard

The transparent result dashboard presents the advisory output in a structured, step-by-step format immediately following each completed diagnostic session. The dashboard displays the reported symptom category, the probable internal cause, the advisory action category, a confidence label, a plain-language explanation of the diagnostic reasoning, the system profile details that informed the interpretation, the warning signs reference table, and the recommended next steps.

The purpose of the result dashboard is to make the system's reasoning fully visible to the user so that they leave the session with a clear, justified basis for their next decision — whether that decision is to monitor, maintain, troubleshoot, or seek professional help. A user who understands the connection between their reported Windows-observable symptoms, their system profile, the probable internal cause identified, and the basis for the recommended action is better equipped to act on that guidance with confidence than one who receives a result without explanation (Kostopoulos et al., 2024; Ravi et al., 2025). The dashboard is therefore the primary mechanism through which the system delivers its decision-support function and through which the improvement in user decision confidence is expected to occur.

Feature 6: Session History and Recurring Pattern Detection Module

The session history module stores each completed diagnostic session locally on the user's device using a SQLite database. Each record includes the system profile snapshot, symptom inputs, diagnostic result, advisory action category, recommendation, plain-language explanation, confidence label, and timestamp. Users access their session history through a dashboard that displays prior sessions in chronological order.

When the same symptom type or internal cause category appears across two or more sessions associated with the same system profile, the system flags it as a recurring pattern and adjusts the advisory output and action category accordingly. A monitoring recommendation appropriate after a first isolated incident may be escalated to a Troubleshoot or Seek Professional Help recommendation if the same Windows-observable symptom recurs across sessions — ensuring that the system's decision support accounts for the user's longitudinal symptom history and not just the most recent session. This

longitudinal capability distinguishes the proposed system from all existing consumer diagnostic tools listed in the comparison table, which treat each user interaction as independent and provide no mechanism for tracking whether a problem is recurring, worsening, or consistently implicating the same internal cause.

PART 5: Discussion

Scope

The proposed system is scoped exclusively to individual consumer desktop PC owners running Windows. It does not address laptops, mobile devices, servers, workstations, or enterprise-managed hardware environments. The target user population consists of students, gamers, beginner PC builders, budget desktop owners, content creators, and general home users — groups that commonly encounter Windows-observable performance symptoms but lack the technical background to diagnose them independently or to decide with confidence whether a problem is within their ability to resolve.

The system is scoped to functional or partially functional desktop PCs exhibiting Windows-observable symptoms. This encompasses systems that can boot fully or partially, display error or warning messages, freeze, crash, slow down, or exhibit Windows-reported S.M.A.R.T. storage health indicators. Fully non-functional systems — those showing no power, no fan activity, no system indicator, and no observable behavior — fall outside the full diagnostic workflow. For these cases, the system provides only a basic boot-status triage checklist and a direct recommendation to seek professional inspection, so the user still leaves with a clear, confidence-supporting decision rather than an unresolved question.

All advisory output is bound to findings derived from user-reported, Windows-observable symptoms and system profile data. The system does not evaluate physical hardware condition, recommend component replacement, or provide upgrade path guidance. Cases requiring physical inspection, voltage measurement, electrical continuity testing, or component substitution fall outside the scope of the advisory engine, and the system is designed to recognize these boundaries by routing such cases to the Seek Professional Help action category with a plain-language explanation of why that category applies.

Limitations

The most significant limitation of the proposed system is its dependence on the accuracy of user-supplied system profile information and symptom responses. If a user enters incorrect OS version information, driver version details, or selects symptom descriptors that do not accurately reflect what they are experiencing, the diagnostic interpretation, action category, and resulting decision support will reflect those inaccuracies. Guided input forms, dropdown selections, contextual prompts, and optional auto-detection features are designed to reduce the likelihood of erroneous entries, but manual input errors cannot be eliminated.

The diagnostic engine is also bound by the scope of the diagnostic rules, expert-validated scenarios, and symptom knowledge base developed during the project. Not every system configuration or failure condition can be anticipated. The system is designed to recognize the limits of its scope and respond by recommending professional inspection rather than speculating beyond the available structured input data — ensuring that the user always receives a usable, confidence-supporting decision even when the system reaches the boundary of its advisory coverage.

Because the system operates as a decision-support tool rather than a diagnostic instrument, all outputs are probabilistic rather than deterministic. This distinction is communicated to users through the plain-language framing of advisory outputs, confidence labels on the result dashboard, and explicit interface statements that results represent probable findings requiring user judgment and, where appropriate, professional confirmation.

A further limitation is the scope of the usability evaluation. A sample of at least 10 participants is sufficient to identify major usability issues and produce a meaningful SUS score and decision confidence comparison (Faulkner, 2003; Virzi, 1992), but it does not support generalization to the full range of desktop PC users across varying technical backgrounds, system configurations, and problem types. Evaluation results should therefore be interpreted as evidence of prototype-level usability and decision confidence impact, not as validated population-level findings.

Expected Contribution

The proposed system contributes a unified, brand-agnostic consumer-facing decision-support workflow that does not currently exist in the available toolset for individual

desktop PC owners. Existing brand-specific utilities provide automated diagnostic functions only within their own ecosystems. Standalone monitoring tools present raw readings without guided interpretation or action classification. General-purpose AI tools provide interpretation without persistent system profile awareness, session continuity, or measurable impact on user decision confidence. No existing tool integrates structured intake of user-reported, Windows-observable symptoms, system-aware internal diagnostic interpretation, four-category action-classified recommendations including a professional inspection pathway, transparent output explanation, warning signs reference, and longitudinal session history within a single downloadable consumer application available to any Windows desktop user.

The system's practical contribution is its potential to reduce uninformed and unconfident decisions among the target user population. By providing structured, system-aware, action-classified guidance that accounts for both the Windows-observable symptom pattern and the user's actual system profile, the application may prompt timely protective actions — such as data backup when S.M.A.R.T. storage health indicators appear across multiple diagnostic sessions — prevent unnecessary professional service expenditure for problems resolvable through software-layer action, and give users a clear, justified basis for seeking professional inspection when their symptom pattern has exceeded what software-layer action can address.

The academic contribution of this study is the application of decision-support design principles — specifically the combination of structured input, system profile context, AI-assisted plain-language explanation, four-category action classification, warning signs escalation guidance, longitudinal session tracking, and pre/post measurement of user decision confidence — to the consumer PC diagnostic domain. These principles have been validated in clinical decision support and enterprise fault diagnosis contexts but have not been systematically applied to individual desktop PC Windows-observable symptom interpretation and advisory guidance with decision confidence as a measured outcome (Kostopoulos et al., 2024; Ravi et al., 2025). The study will produce an evaluated prototype, an expert-validated diagnostic scenario set, and empirical data on usability and user decision confidence that can serve as a methodological foundation for future research in consumer-facing hardware advisory systems.

PART 6: Traceability Matrix

RRL Finding / Theme	Identified Gap	Research Question	Proposed Feature / Design Decision
Ravasio, Schär, & Krueger (2004): Desktop PC users report performance problems inconsistently and rely on fragmented, non-personalized sources when seeking hardware guidance.	Desktop PC users lack a structured mechanism to report Windows-observable performance symptoms consistently and connect those symptoms to probable internal causes in a repeatable, evaluable way.	To what extent does the structured Windows-observable symptom intake and system profile module collect relevant information needed to classify observable desktop PC problems into defined internal symptom categories?	Structured Windows-Observable Symptom Intake Module: A guided diagnostic session using controlled questions covering symptom type, affected Windows-environment activity, frequency, severity, duration, recent changes, system state, and Windows-reported warning signs.
Dahouk & Abu-Naser (2018): Rule-based knowledge systems can map PC symptoms to probable component-related causes, but existing implementations lack persistent hardware profile context and user-accessible plain-language explanation.	Consumer-facing diagnostic tools do not provide a unified symptom-based workflow that maps user-reported, Windows-observable symptoms to probable internal causes within the context of the user's specific system configuration.	How accurately does the internal diagnostic interpretation engine map user-reported, Windows-observable symptoms and system profile data to probable internal cause categories?	Internal Diagnostic Interpretation Engine: A diagnostic engine that analyzes OS symptom inputs and system profile data to identify probable internal cause categories across four primary categories and supporting categories.

Abu-Rasheed, Weber, & Fathi (2023): Context-based advisory systems produce substantially more relevant recommendations when user profile data is incorporated into interpretation logic.	PC monitoring and diagnostic tools do not adapt their advisory output based on the user's full system profile and specific reported Windows-observable symptom context.	How accurately does the internal diagnostic interpretation engine map user-reported, Windows-observable symptoms and system profile data to probable internal cause categories?	System Profile Module: A module that stores or assists in collecting CPU model, RAM capacity, storage type and capacity, OS version, driver versions, and system age so that all diagnostic outputs are grounded in the user's actual configuration.
Ravi et al. (2025): AI-driven decision support systems produce more actionable outputs when analytical findings are converted into defined action categories rather than presented as raw conclusions.	Existing PC diagnostic and monitoring tools do not convert their findings into structured action categories. Users receive status indicators but no classification of what those findings require them to do.	How relevant and appropriate are the system-generated advisory actions when evaluated against expert-reviewed expected recommendations for the same diagnostic scenario?	Advisory Action Module: A recommendation module that classifies every output into one of four practical action categories — Monitor, Maintain, Troubleshoot, or Seek Professional Help — with a plain-language explanation and Windows-observable escalation warning signs reference table.
Kostopoulos, Davrazos, & Kotsiantis (2024): Explainable AI decision support requires that reasoning behind	Desktop PC users who receive a diagnostic result without explanation are unlikely to understand, evaluate,	How clearly does the system explain the connection between reported symptoms, system profile,	Transparent Result Dashboard: A result page presenting the reported symptom category, probable

recommendations be communicated clearly. Systems that provide results without explanation are less trusted and less acted upon by non-expert users.	or act on it with confidence.	probable internal cause, and recommended action?	internal cause, advisory action category, confidence label, plain-language diagnostic reasoning, system profile details considered, Windows-observable warning signs table, and recommended next steps.
Zhang, Yang, & Wang (2019): Data-driven fault diagnosis methods that combine structured input data with system context and rule-based logic produce more reliable outputs than approaches relying on unstructured self-report alone.	Consumer desktop PC users do not have access to expert-level interpretation of Windows-observable symptoms, particularly when the same observable symptom may originate from different internal causes depending on the user's specific system configuration.	How accurately does the internal diagnostic interpretation engine map user-reported, Windows-observable symptoms and system profile data to probable internal cause categories?	AI-Assisted Diagnostic Explanation Layer: AI assistance used to generate plain-language diagnostic explanations within a controlled framework of structured OS symptom inputs, system profile data, rule-based diagnostic mapping, and expert-validated scenarios.
Holmes-Rovner et al. (1999); Alter (1978): Decision support systems improve user decision confidence and	No existing consumer PC diagnostic tool incorporates longitudinal session tracking or recurring	To what extent does the session history and recurring pattern detection module help users	Session History & Recurring Pattern Detection Module: A module that saves each completed

consistency when they provide structured outputs across multiple interaction instances rather than treating each interaction as independent.	pattern detection, providing no mechanism for identifying whether a problem is recurring, worsening, or repeatedly implicating the same internal cause.	identify persistent symptom patterns and repeated internal cause findings across multiple diagnostic sessions?	diagnostic session locally, displays prior sessions in chronological order; and flags recurring Windows-observable symptom types or internal cause categories across two or more sessions.
--	---	--	--

PART 7: References

- 1.) Alter, S. (1978). Development patterns for decision support systems. *MIS Quarterly*, 2(3), 33–42. <https://doi.org/10.2307/248960>
- 2.) Abu-Rasheed, H., Weber, C., & Fathi, M. (2023). Context-based learning: A survey of contextual indicators for personalized and adaptive learning recommendations. *Education and Information Technologies*, 28(4), 3839–3868. <https://doi.org/10.1007/s10639-022-11051-4>
- 3.) Amato, F., López, A., Peña-Méndez, E. M., Vañhara, P., Hampl, A., & Havel, J. (2013). Artificial neural networks in medical diagnosis. *Journal of Applied Biomedicine*, 11(2), 47–58. <https://doi.org/10.2478/v10136-012-0031-x>
- 4.) Backblaze. (2024). Backblaze drive stats for 2023: Annual hard drive reliability report. <https://www.backblaze.com/blog/backblaze-drive-stats-for-2023/>
- 5.) Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor and Francis.
- 6.) Christodoulou, A., & Angeli, C. (2022). Adaptive learning techniques: A systematic review. *Frontiers in Education*, 7, 893745. <https://doi.org/10.3389/feduc.2022.893745>
- 7.) Dahouk, A. W., & Abu-Naser, S. S. (2018). A proposed knowledge based system for desktop PC troubleshooting. *International Journal of Academic Pedagogical Research*, 2(6), 1–8.

- 8.) Dumas, J. S., & Redish, J. C. (1999). A practical guide to usability testing (Rev. ed.). Intellect Books.
- 9.) Duran, M., Uz, C., Saritepeci, M., & Colak, C. (2022). Cognitive load in technical computing environments. *Computers and Education Open*, 3, 100093. <https://doi.org/10.1016/j.caeo.2022.100093>
- 10.) Faulkner, L. (2003). Beyond the five-user assumption: Benefits of increased sample sizes in usability testing. *Behavior Research Methods, Instruments, & Computers*, 35(3), 379–383. <https://doi.org/10.3758/BF03195514>
- 11.) Goyal, G., Saini, P., & Kumar, R. (2025). AI-driven fault detection in hardware systems. *Journal of Information Systems Engineering and Management*, 10(1), 112–128. <https://doi.org/10.52783/jisem.v10i1.112>
- 12.) Holmes-Rovner, M., Kroll, J., Rovner, D. R., Schmitt, N., Rothert, M., Padonu, G., & Talarczyk, G. (1999). Patient decision support intervention: Increased consistency with decision analytic models. *Medical Care*, 37(3), 270–284. <https://doi.org/10.1097/00005650-199903000-00007>
- 13.) Hwang, G.-J. (2014). Definition, framework, and research issues of smart learning environments: A context-aware ubiquitous learning perspective. *Smart Learning Environments*, 1(1), Article 4. <https://doi.org/10.1186/s40561-014-0004-5>
- 14.) Jha, S., & Topcu, T. (2021). Explainability and auditability in AI: Definitions, tools, and emerging standards. *AI and Ethics*, 1(3), 251–270. <https://doi.org/10.1007/s43681-021-00051-0>
- 15.) Kostopoulos, G., Davrazos, G., & Kotsiantis, S. (2024). Explainable artificial intelligence-based decision support systems: A recent review. *Electronics*, 13(14), 2842. <https://doi.org/10.3390/electronics13142842>
- 16.) Lee, W.-J., Wu, H., Yun, H., Kim, H., Jun, M.-B. G., & Sutherland, J. W. (2019). Predictive quality analytics of surface roughness in CNC machining using support vector machine. *Procedia CIRP*, 80, 114–119. <https://doi.org/10.1016/j.procir.2019.01.072>
- 17.) Mothukuri, V., Parizi, R. M., Pouriyeh, S., Huang, Y., Dehghantanha, A., & Srivastava, G. (2021). A survey on security and privacy of federated learning. *Future Generation Computer Systems*, 115, 619–640. <https://doi.org/10.1016/j.future.2020.10.007>
- 18.) Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.
- 19.) Norman, D. A. (2013). The design of everyday things (Rev. and exp. ed.). Basic Books.
- 20.) Pierce, R. L., Van Biesen, W., Van Cauwenberge, D., Decruyenaere, J., & Sterckx, S. (2022). Explainability in medicine in an era of AI-based clinical decision support systems. *Frontiers in Genetics*, 13, 903600. <https://doi.org/10.3389/fgene.2022.903600>

- 21.) Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.
- 22.) Ravi, M., Negi, A., Bommi, N. S., & Rouf, N. (2025). Evolution of AI-driven decision making with decision support systems, expert systems, recommender systems, and XAI. *IETE Technical Review*, 42(4), 428–465. <https://doi.org/10.1080/02564602.2024.2447840>
- 23.) Ravasio, P., Schär, S. G., & Krueger, H. (2004). In pursuit of desktop evolution: User problems and practices with modern desktop systems. *ACM Transactions on Computer-Human Interaction*, 11(2), 156–180. <https://doi.org/10.1145/1005361.1005363>
- 24.) Rubin, J., & Chisnall, D. (2008). Handbook of usability testing: How to plan, design, and conduct effective tests (2nd ed.). Wiley.
- 25.) Sauro, J., & Lewis, J. R. (2016). Quantifying the user experience: Practical statistics for user research (2nd ed.). Morgan Kaufmann.
- 26.) Skulmowski, A., & Xu, K. M. (2021). Understanding cognitive load in digital and online learning: A new perspective on extraneous cognitive load. *Educational Psychology Review*, 34(1), 171–196. <https://doi.org/10.1007/s10648-021-09624-7>
- 27.) Soper, M. E. (2014). The PC and gadget help desk: A do-it-yourself guide to troubleshooting and repairing. Que Publishing.
- 28.) Sweller, J., van Merriënboer, J. J. G., & Paas, F. (2019). Cognitive architecture and instructional design: 20 years later. *Educational Psychology Review*, 31(2), 261–292. <https://doi.org/10.1007/s10648-019-09465-5>
- 29.) Tsai, C.-H., Cheng, C.-H., Yeh, D.-Y., & Lin, H.-H. (2016). Intelligent tutoring systems and context-aware learning in mobile environments. *Journal of Educational Technology and Society*, 19(2), 220–234.
- 30.) Virzi, R. A. (1992). Refining the test phase of usability evaluation: How many subjects is enough? *Human Factors*, 34(4), 457–468. <https://doi.org/10.1177/001872089203400407>
- 31.) Weiss, G. M., & Provost, F. (2003). Learning when training data are costly: The effect of class distribution on tree induction. *Journal of Artificial Intelligence Research*, 19, 315–354. <https://doi.org/10.1613/jair.1199>
- 32.) Zhang, W., Yang, D., & Wang, H. (2019). Data-driven methods for predictive maintenance of industrial equipment: A survey. *IEEE Systems Journal*, 13(3), 2213–2227. <https://doi.org/10.1109/JSYST.2019.2905565>

- 33.) Zheng, Y., & Hu, L. (2024). AI prospects in power system fault diagnosis: A review of methods and applications. *International Core Journal of Engineering*, 10(3), 54–67.
[https://doi.org/10.6919/ICJE.202403_10\(3\).0008](https://doi.org/10.6919/ICJE.202403_10(3).0008)